Lecture 10

File Systems; Protection

Mingyu Gao

gaomy@tsinghua.edu.cn

IIIS, Tsinghua

Recall: Hardware Interface of Storage Devices

- Secondary storage devices (e.g., disks) are accessed in units of blocks (e.g., 4 kB in Unix), or pages in SSDs
  - A block is a logical data transfer unit; a sector is the physical unit
  - A block is one or multiple sectors

- We now assume logical block address (LBA) for each block in a disk
  - Sequential, linear: from 0 to max number of blocks
  - May differ from physical positions: blocks with bad sectors are skipped automatically by disk controller; pages are wear-leveled in SSDs
  - Disk/SSD controller translates from LBA to physical positions, i.e., hardware shields OS from internal disk structures

File Systems

- File system: a component in OS that transforms block interface into high-level interface like files and directories

- User's view: files, as durable data structures
- Syscall interface (between user and OS): collection of contiguous bytes
  - Read/write the next few bytes, or a specific range of bytes (e.g., bytes 100 – 200)
  - Oblivious to what kinds of data are stored
- OS internal view: collection of blocks (may be incontiguous)
  - Data in file systems are always accessed and allocated in whole blocks
  - To access a byte, first find the block, read the block into memory, read/write the bytes in memory, and writeback to disk (if written)
    - Cannot directly read or write a byte in disk

Files Are Named

- Files must be stored durably, i.e., persist even after the creating process is terminated and even after system is shutdown

- Files are named, and referred by the names
  - E.g., `example.cpp`, `readme.txt`, `notepad.exe`
  - Easier to be recognized than addresses
- When a file is named, it becomes independent of the process, the user, and even the system that created it
  - A user/process can read and modify a file created by another user/process, as long as using the same file name
  - A file can be transferred between systems, e.g., over network or in a USB drive, and the file name keeps the same

File Systems To Users

- From a user's perspective, a file is the smallest granularity of logical secondary storage; that is, data cannot be written to secondary storage unless they are within a file

- Unix, Linux, and some other OSes have a `/proc` file system that uses file-system interface to access system information (e.g., process details)
  - Read a system state → read from a file
  - Change a system configuration → write to a file
  - E.g., `/proc/cpuinfo`, `/proc/567/`, `/proc/sys/`

File System Components

- A file system mainly consists of two parts
  - A collection of files, each storing relevant data in a group of disk blocks
  - A directory structure, organizing files and mapping names to files

- Disk management: organize disk blocks into files
  - Track which blocks contain data for which file: where to access a file on disk
  - Track free blocks on disk: where to put newly written data
- Naming: be able to find files by names, not by blocks
  - Track what files are in a directory: find a file given its name
- Protection: keep different users' data isolated
- Reliability/durability: keep data consistent despite crashes, failures, etc.
  - E.g., transactions (discussed in later lectures)

Outline

- File basics
  - Metadata: file descriptor, a.k.a., inode, file control block
  - Operations: open/close, read/write/seek/sync
- Data block organization: which blocks for which file
  - Contiguous, linked, indexed
- Free space management: track free blocks
  - Linked lists, bitmap
- Directory and links: naming
- Performance optimization: buffer cache
- Protection: authentication and authorization

File Descriptors (a.k.a., inode)

- File descriptor: OS data structure about a file's metadata information
  - Stored on disk along with file data: metadata must also be persisted
  - Kept in OS kernel memory when file is open: for convenient operations
- Called inode in Unix-style systems, or sometimes as file control block

- Metadata in a file descriptor, i.e., file attributes
  - File size
  - File location, e.g., sectors on disk
  - Access timestamps, e.g., create, last read, last write
  - Protection info: e.g., access permissions (R/W/X), owner ID, group ID, etc.
    - More on this part later
  - ……

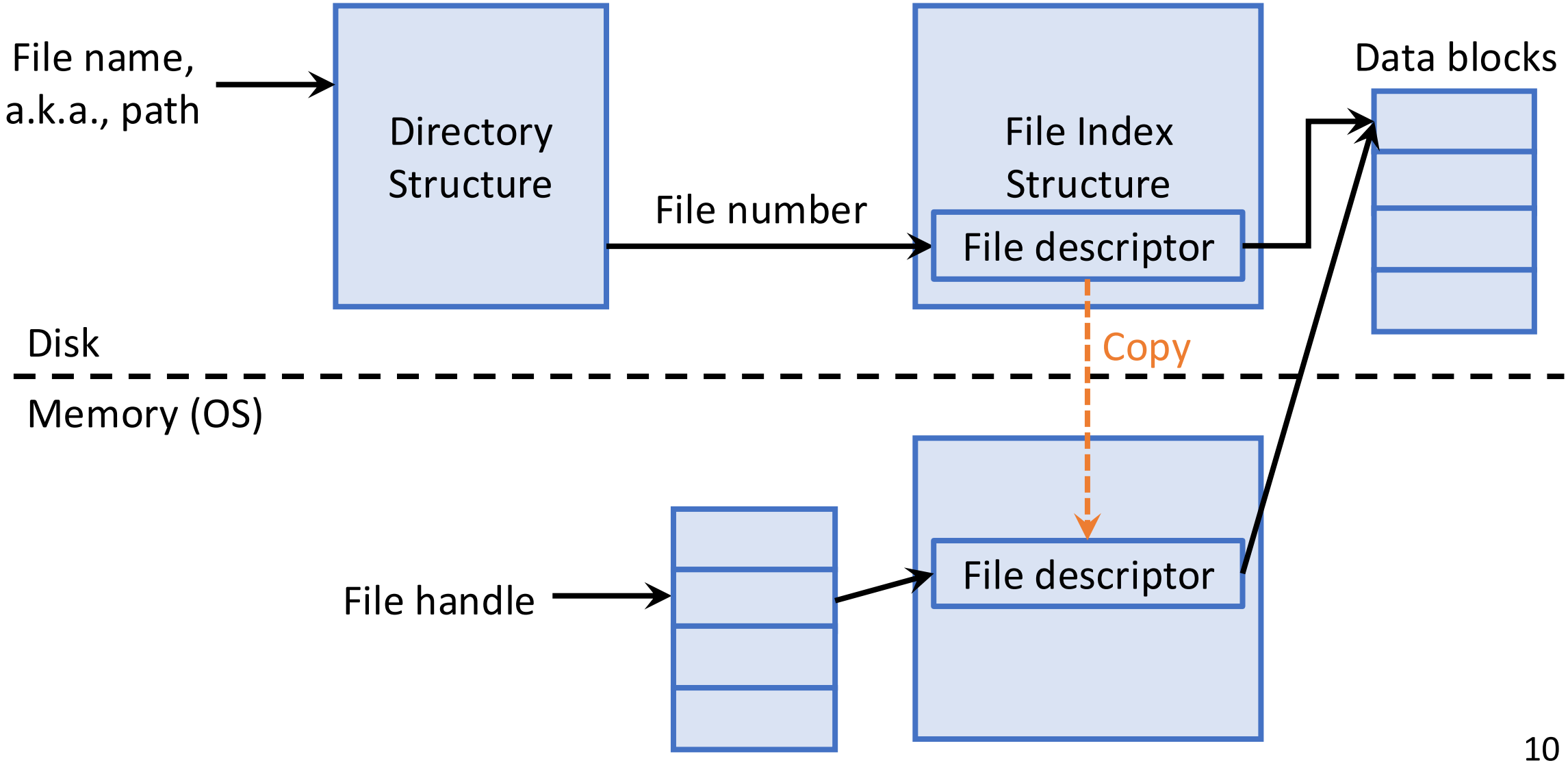
File Operations

- Open: use name to search a file in the directory, i.e., resolve file name
  - Translate file name into a file number (i-number in Unix), which is used to locate the file descriptor (inode in Unix) on disk
    - More details on this later
  - Copy file descriptor from disk to OS kernel memory space
  - Return a file handle (usually an integer) to user process
    - Q: Why not directly return (a pointer to) the file descriptor?
  - Distinguish: file number (i-number) vs. file descriptor (inode) vs. file handle

- Later read/write/seek/sync operate on file handles
  - OS internally uses file handle to locate file descriptor, and to find data blocks

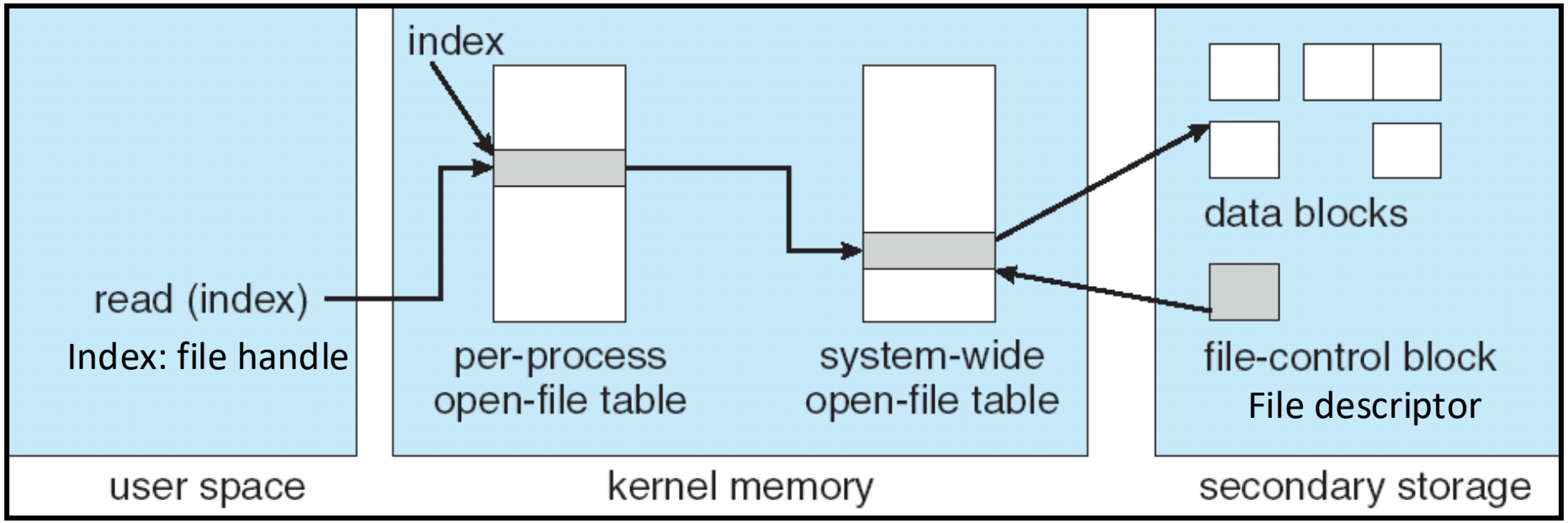
- Close: when no longer operating on the file

File System Components



In-Memory File System Structures

- Two levels of open-file tables: per-process and system-wide
  - Multiple processes may open the same file at the same time
  - The second process simply adds an entry to its per-process table, pointing to the already existing entry in the system-wide table
  - System-wide table entries have reference counters: automatically remove entry when the last process closes the file



File Access Patterns

- Sequential: data are processed in order, one byte after another
  - Most common, e.g., when parsing a file line by line or reading in all file content
- Random: can address any byte in the middle of a file directly
  - With the seek ability (different from disk head seek)
  - E.g., demand paging from swap file, databases
  - Should be better than naïve – not have to read all predecessor bytes
- Keyed (or indexed): search for blocks with particular contents
  - I.e., find all files with the word "system"
  - E.g., hash table, associative database, dictionary
  - Most OSes do not support this; instead, build a database (later lectures)
    - Example in OS: MacOS Spotlight

Outline

- File basics
  - Metadata: file descriptor, a.k.a., inode, file control block
  - Operations: open/close, read/write/seek/sync
- Data block organization: which blocks for which file
  - Contiguous, linked, indexed
- Free space management: track free blocks
  - Linked lists, bitmap
- Directory and links: naming
- Performance optimization: buffer cache
- Protection: authentication and authorization

Data Block Organization

- How to allocate disk blocks of a file?
  - Similar to memory allocation
- Recall: file data are always in full blocks
  - E.g., a file of 15000 bytes would be allocated with 4 blocks of 4 kB each
  - Wasted space: internal fragmentation
- Design issues to consider
  - Most files are small (a few kB or less): per-file overheads must be low
  - Most of the disk space is in large files
  - Performance must be good for large files even for random accesses
  - File sizes may grow unpredictably over time

Characteristics of Files

- Left: most files are small, growing numbers of files
- Right: most of the space is occupied by the rare big files
  - Big files are more and more common (e.g., multimedia); see the growing second peak in the figure

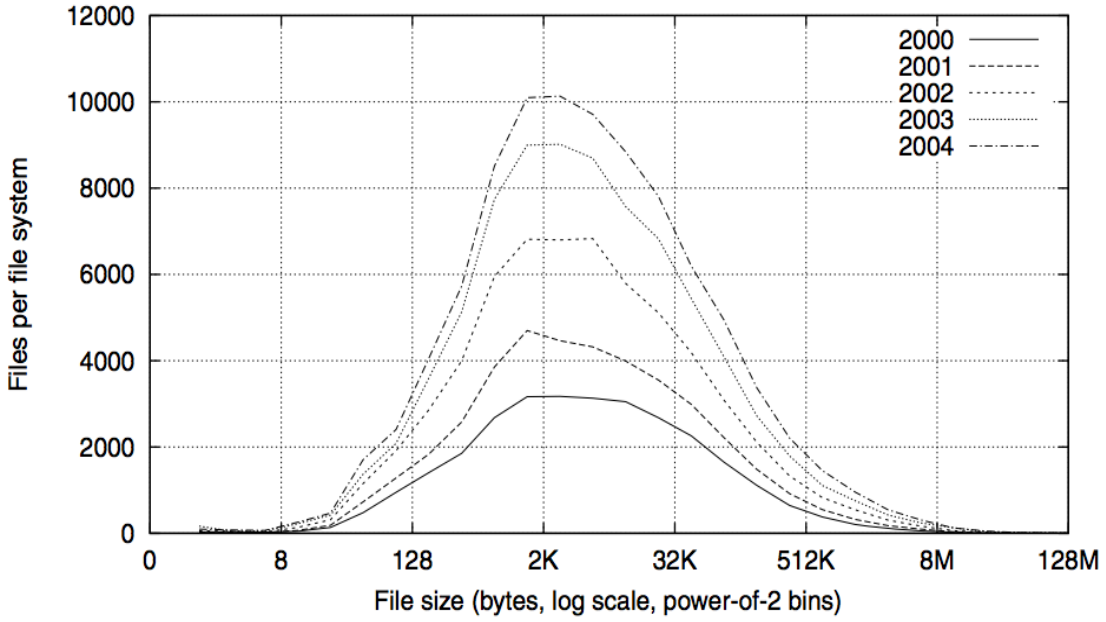


Fig. 2. Histograms of files by size.

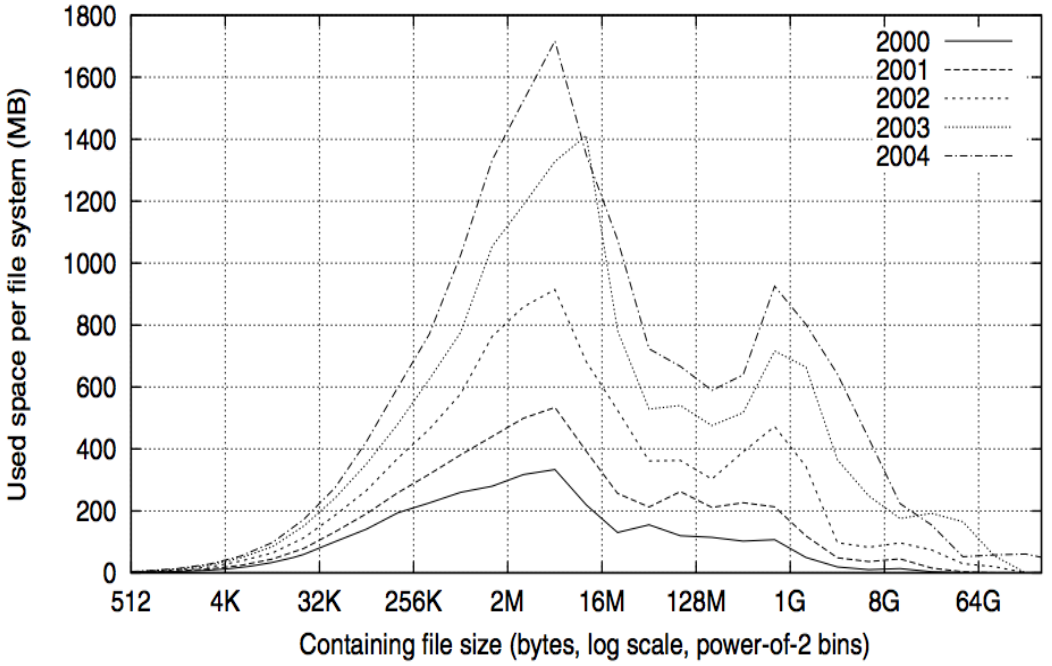


Fig. 4. Histograms of bytes by containing file size.

N. Agrawal, et al. *A Five-Year Study of File-System Metadata*. FAST, 2007.

Data Block Organization 1: Contiguous

- Simplest approach: contiguous allocation
  - Similar to a memory allocator for dynamic heap allocation
  - Given the size of a file, allocate contiguous blocks all at once
  - Block location and counts are stored in file descriptor

- Pros
  - Simple
  - Easy for both sequential and random accesses
  - Few disk seeks with good performance
- Cons
  - Harder and harder to allocate large files (due to external fragmentation)
  - Hard to grow file sizes

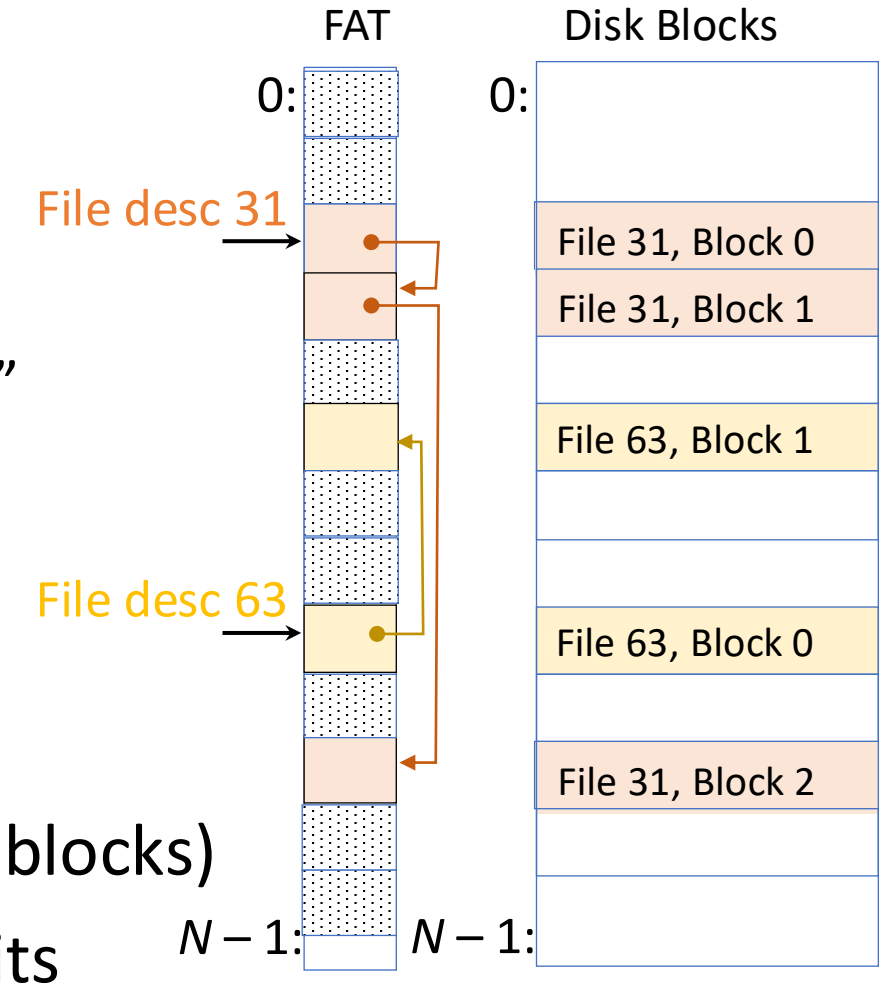
Data Block Organization 2: Linked Files

- Linked files: keep a linked list of all blocks of a file
  - In file descriptor, only need to keep a pointer to the first block
- To randomly access a specific position
  - From file descriptor, follow linked list until reaching the block

- Pros
  - No external fragmentation; high utilization of disk space
  - Easy to allocate new blocks; file size can grow, and may even grow in the middle
- Cons
  - More disk seeks, even for sequential accesses (poor locality)
  - Overheads of tracking pointers

Example: Windows FAT

- Like linked allocation, except not to keep links in data blocks themselves
- Maintain a separate FAT (File Allocation Table)
  - FAT is copied into memory during normal operation
  - Each FAT entry 1-1 corresponds to each block
  - Each FAT entry stores index of next block in file
  - Special values for “last block in file” and “free block”
- File descriptor stores index of first block
- Q: What happens when we format/quick-format a disk?

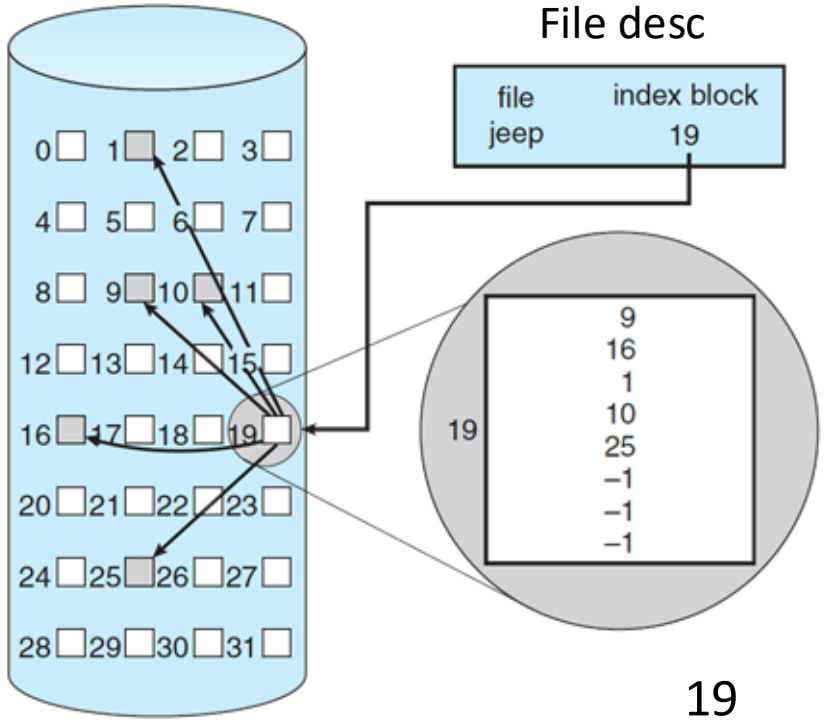


- Originally, each FAT entry was 16 bits (only 2^{16} blocks)
- FAT32 supports larger disks; each entry is 28 bits

Data Block Organization 3: Indexed Files

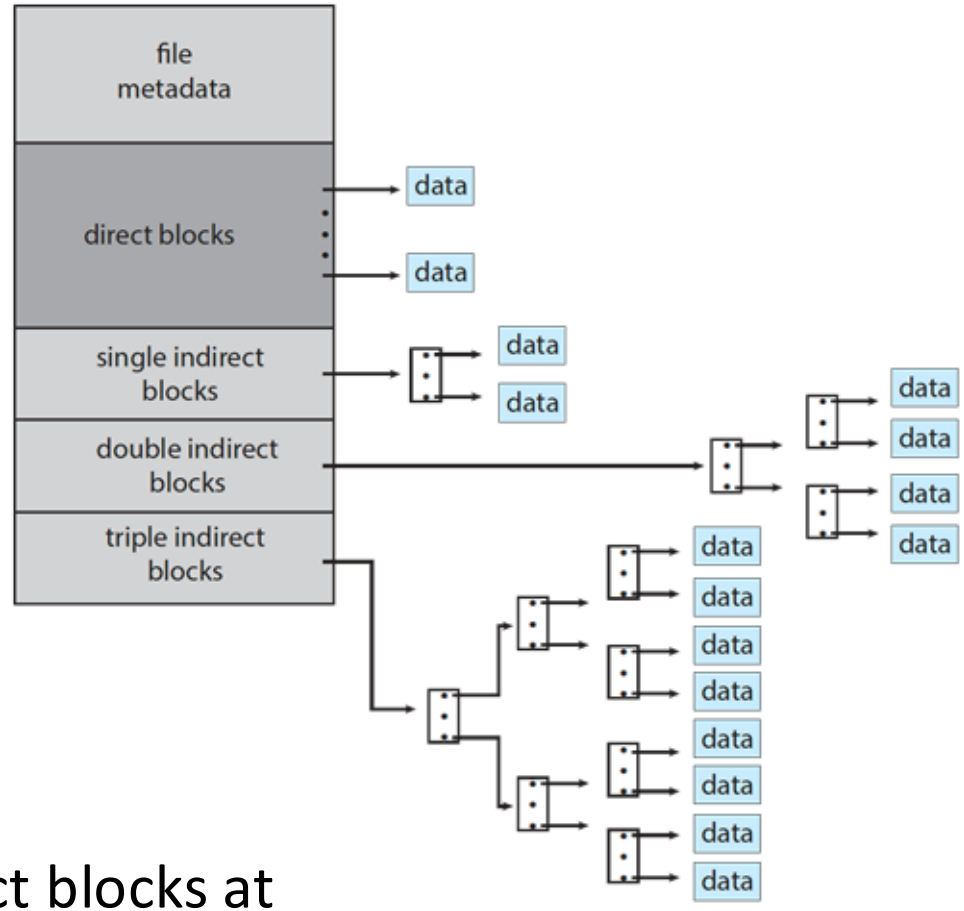
- Indexed files: keep the set of pointers to data blocks
  - Allocate a specific space (e.g., contiguous blocks) to store pointers to all blocks
  - Initially all pointers are null; only allocate actual data blocks as file is written
  - Number of pointers (and thus max file size) is usually pre-determined and fixed

- Pros: have pros of both contiguous and linked
  - Support direct access; fast random access
    - Locating the target block pointer is fast; no need to traverse long linked list
  - No external fragmentation
- Cons
  - Still some amount of seeks
  - File size is limited, or, many unused pointers



Example: 4.3 BSD Unix – Multi-Level Index

- Blocks are 4 kB; each pointer is 4 bytes
- File descriptor has 14 block pointers
  - First 12 point to data blocks (direct blocks)
  - Next one points to an indirect block, which contains 1024 pointers to data blocks
    - Indirect block is not allocated until needed
  - Next one points to a double-indirect block
- Advantages
  - Small files have fast access (using direct blocks)
    - Up to 48 kB for files with only direct blocks
  - Support large files reasonably fast (using indirect blocks at the cost of a few more seeks)
    - Roughly up to 4 MB when using indirect blocks, up to 4 GB when using double-indirect blocks



Free Space Management

- Many early systems use a linked list of free blocks
  - Similar to free space tracking in dynamic heap allocators

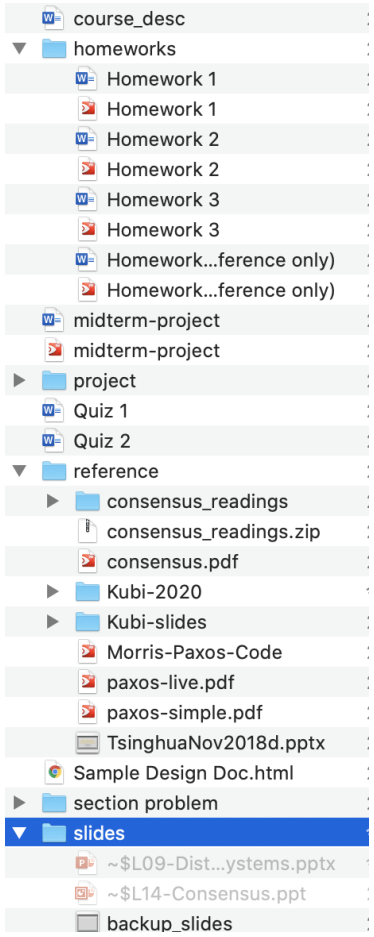
- 4.3 BSD Unix approach: bitmap
  - Keep an array of bits, one per block: 1 means free, 0 means in use
  - During allocation, search for a block close to the previously last block of the file, so that seek distance is minimized and locality is improved
    - When disk is not full, this works quite well
    - When disk is full, search takes long time and locality is not improved
  - Solution: do not let disk fill up!
    - Reserve 10% capacity
    - If disk gets 90% full, tell user it is full and disallow further data store
    - Space vs. time tradeoff

Outline

- File basics
  - Metadata: file descriptor, a.k.a., inode, file control block
  - Operations: open/close, read/write/seek/sync
- Data block organization: which blocks for which file
  - Contiguous, linked, indexed
- Free space management: track free blocks
  - Linked lists, bitmap
- Directory and links: naming
- Performance optimization: buffer cache
- Protection: authentication and authorization

Directory

- Directory is used to map names to file numbers
  - Recall: file number is the index of file descriptors on disk
- Early (simple but functional) directory management
  - A single directory for the entire disk
    - If one user uses a file name, no one else can use any more
  - A single directory for each user
    - Still hard to organize information
- Modern systems support hierarchical directory structures
  - Each directory is a collection of files and other directories
  - Names have slashes "/" separating the levels, i.e., a path

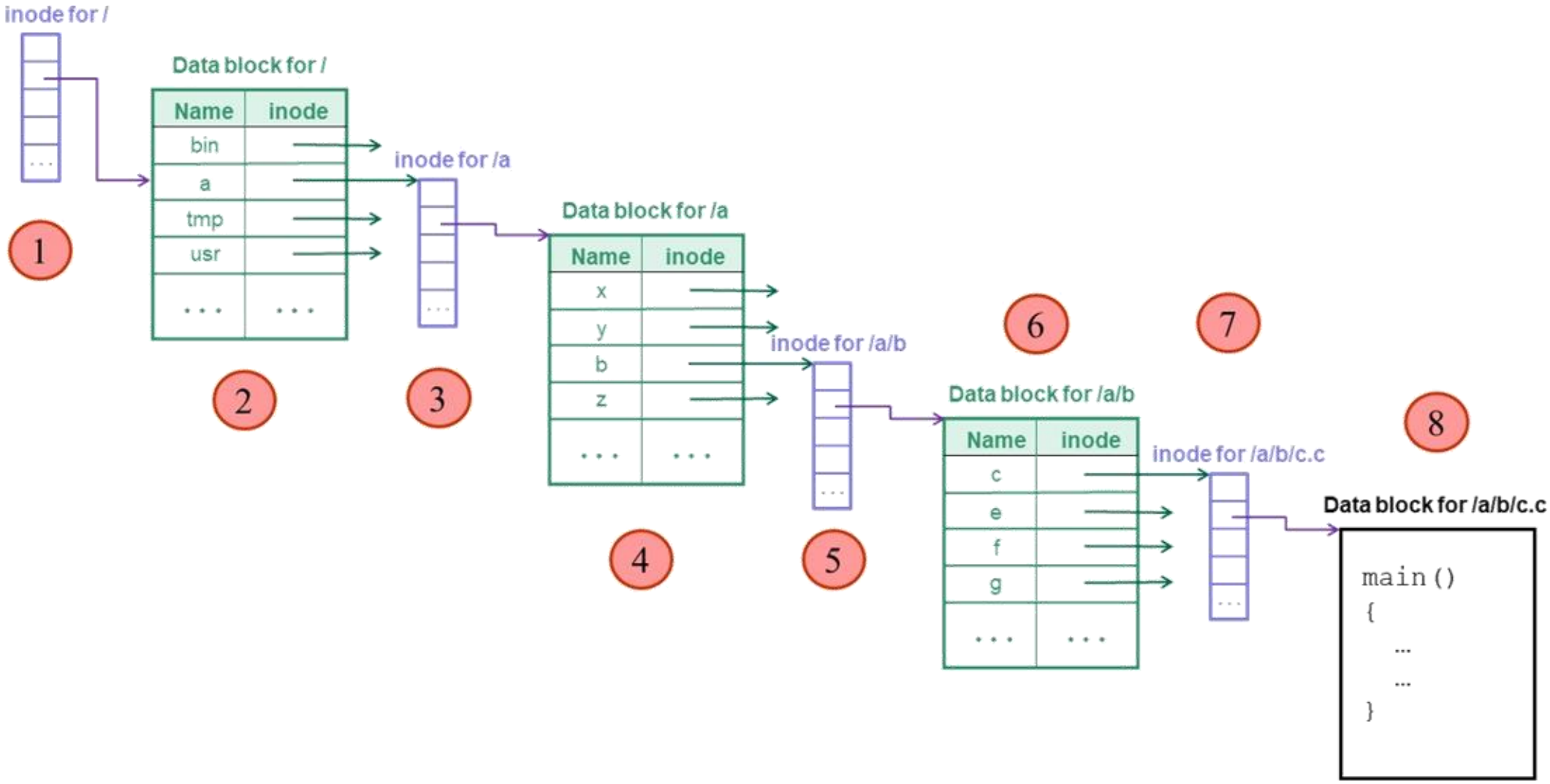


Unix/Linux Directory Structures

- Directories are just like regular files, e.g., with a file descriptor, except that file descriptor has a special bit indicating it is a directory
- "Data" of a directory is an unordered list of <name, pointer> pairs
  - Each pair is for a child file or a sub-directory; pointer is a file number that points to its file descriptor
- A special directory called the root has no name, and has i-number of 2
  - Fixed location on disk
  - i-number 0 is null; i-number 1 means bad blocks

- User programs can read directories just like regular files
  - E.g., on Linux, try `vim /tmp`
- Usually only OS can write directories

Directory Structure Example



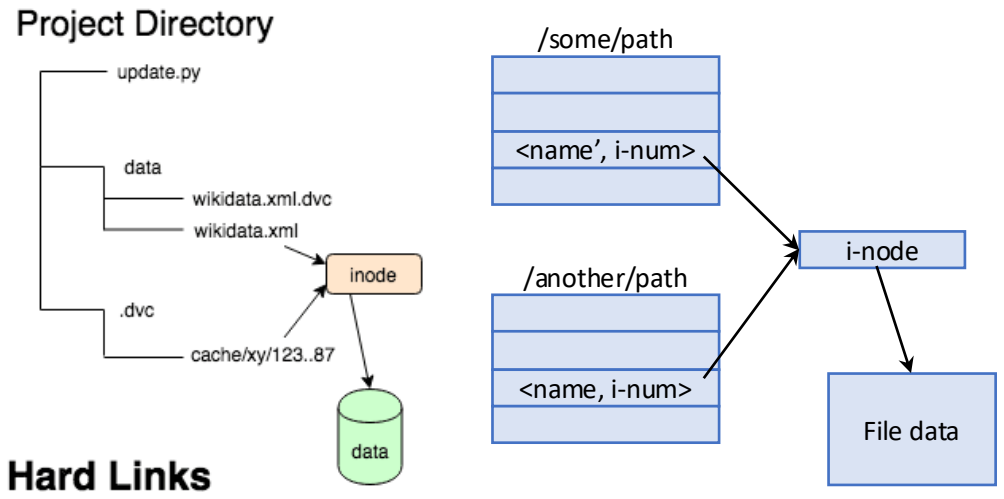
Directory Access Example

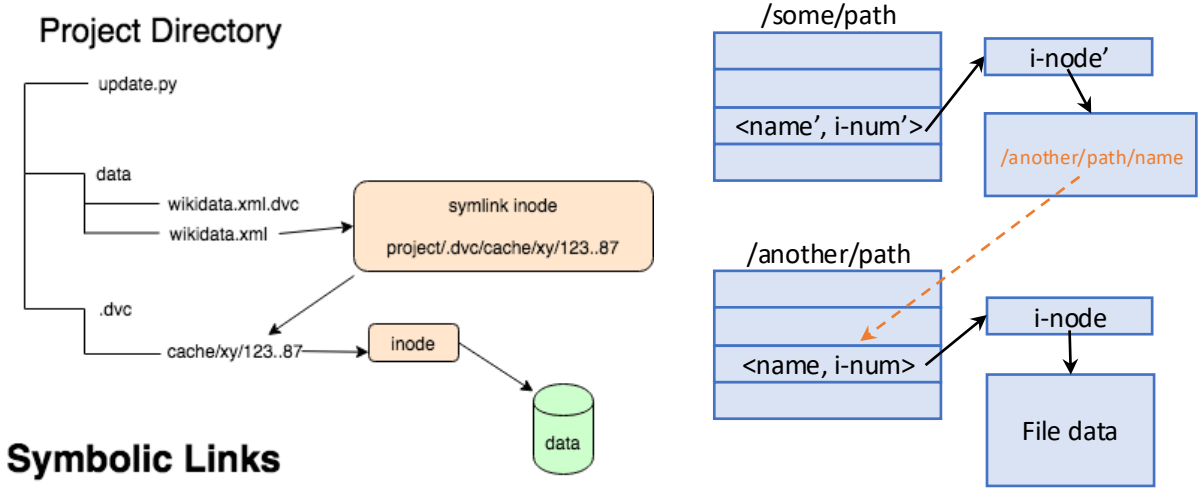
- How many disk accesses to resolve `/my/book/count`?

- Read in inode for root (fixed location on disk)
- Read in first data block for root; search for `my`
  - A list of file name/index pairs; search linearly
- Read in inode for `/my`
- Read in first data block for `/my`; search for book
- Read in inode for `/my/book`
- Read in first data block for `/my/book`; search for `count`
- Read in inode for `/my/book/count`

Links

- Hard link: set another directory entry to the file number for a file
  - Create another name (path) for the same file
  - Make directory structure a DAG (directed acyclic graph), not exactly a tree
  - Q: When can a file be deleted? How?
- Soft or symbolic link: special file whose content is another name (path)
  - Stored as regular files, but with a flag set in file descriptor





Current Working Directory

- Cumbersome to require to always specify full path name
- Make OS remember a special directory per process: working directory
  - The inode of the working directory is kept in the PCB (process control block)
  - If a path starts without “/”, then look up starting from the working directory
  - Paths starting with “/” are looked up starting from the root
- Other special directory entries: “.”, “..”

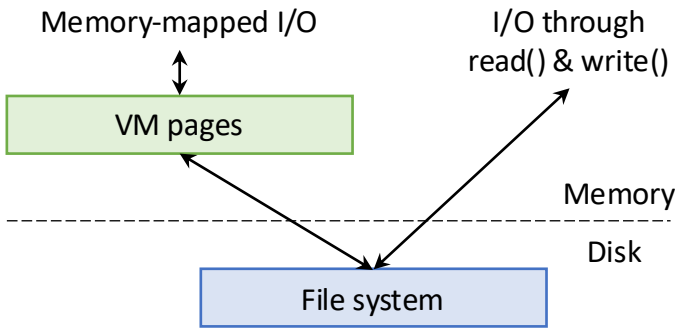
Outline

- File basics
  - Metadata: file descriptor, a.k.a., inode, file control block
  - Operations: open/close, read/write/seek/sync
- Data block organization: which blocks for which file
  - Contiguous, linked, indexed
- Free space management: track free blocks
  - Linked lists, bitmap
- Directory and links: naming
- Performance optimization: buffer cache
- Protection: authentication and authorization

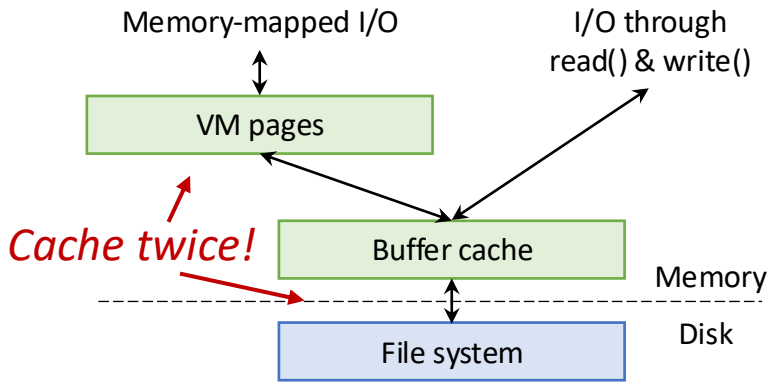
Buffer Cache

- Use part of main memory to retain recently accessed disk blocks
- Implemented entirely in OS software; can cache inodes, data blocks for directories and files, indirect blocks, free bitmaps, etc.
- Managed in unit of blocks; replacement policy is usually LRU
- When hit, avoid slow access to disk

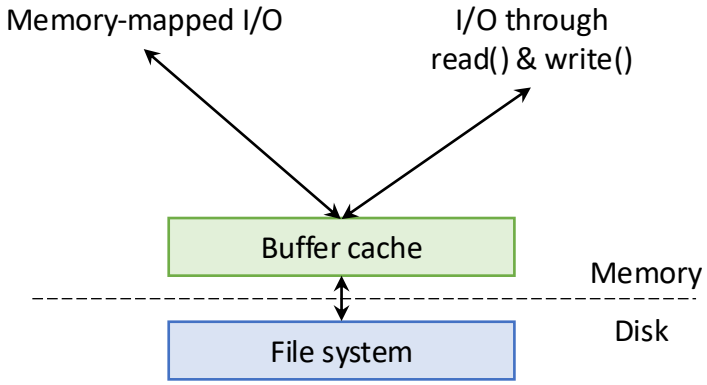
Buffer Cache and Virtual Memory



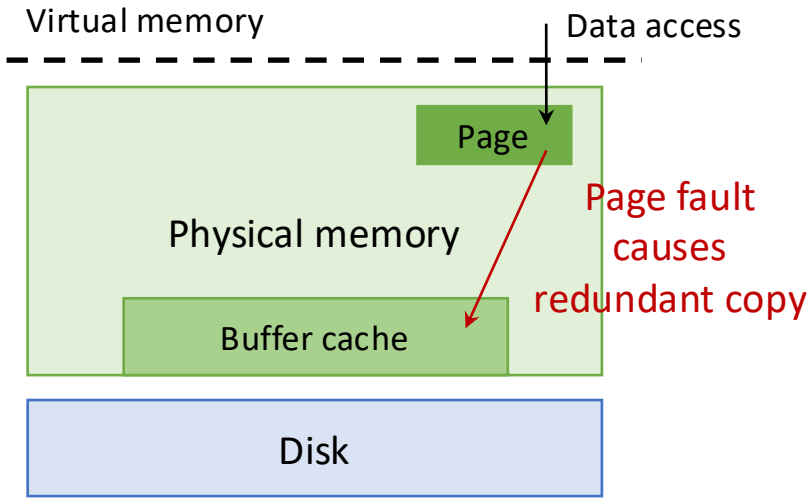
Without cache



With buffer cache



With unified cache



- Many OSes now unify buffer cache and VM page pool, in order to avoid double caching issue for memory-mapped files

Synchronous vs. Delayed Writes

- What happens when a block in the buffer cache is modified?

- Synchronous writes: immediately write through to disk
  - Safe; no data lost
  - Slow
  - May be unnecessary if there are many writes to one block
- Delayed writes: do not immediately write to disk, e.g., wait for a while
  - Fast; writes complete immediately after memory writes
  - Dangerous: may lose data if system crashes
    - See `sync()` syscall in UNIX
  - Sometimes difficult to decide the delay

Outline

- File basics
  - Metadata: file descriptor, a.k.a., inode, file control block
  - Operations: open/close, read/write/seek/sync
- Data block organization: which blocks for which file
  - Contiguous, linked, indexed
- Free space management: track free blocks
  - Linked lists, bitmap
- Directory and links: naming
- Performance optimization: buffer cache
- Protection: authentication and authorization

Protection

- Protection: prevent accidental and malicious misuse

- Authentication: identify a responsible party (principal)
  - Who are you?
- Authorization: determine what actions can be performed by a principal on an object
  - What can you do?
- Access enforcement: combine authentication and authorization to control access

Authentication

- Typically done with passwords
  - Passwords should be hard to guess: long and obscure
- System must store passwords of all users; the password storage then become a vulnerability for attack
- Solution: store one-way hash results instead of plaintext passwords
- Problem: dictionary attack, a.k.a., rainbow table attack
  - Only need to build one dictionary to attack *all* users
- Solution: concatenate with random salts, i.e., hash of password|salt
  - Now must build a different dictionary for *each* user, significantly more difficult
  - Salts do not need to be kept secret

- Other authentication method: physical badges, USB key, bio info, …

Attach Principal to Processes

- Once authenticated, e.g., user log in, the identity of the principal, e.g., user ID, is attached to every process executed under that login
  - Each process inherits the user ID from its parent, e.g., through fork
  - Processes are usually spawned in a user shell

- After authentication, the authenticated identity of the principal must be protected from tampering, since other parts of the system rely on it
  - Processes are isolated

Authorization

- Determine which principals can do what operations on which objects
  - E.g., principals are users, objects are files, operations are read/write/execute

- Logically, authorization information can be represented as access matrix
  - One row per principal
  - One column per object
  - Each entry indicates what operations are allowed
- Such a full matrix is too large, and usually highly sparse
  - Each objects are only interesting to a few principals
- We can compress in two ways
  - Access control lists
  - Capabilities

Access Control List (ACL)

- Organized by columns: with each object, store info about which principals are allowed to perform what operations
  - E.g., a list of <user, permission> pairs, as part of its metadata
  - Challenge: size of ACL
  - Solution: users can be organized into groups, and a single ACL entry for a group
- Example: Unix uses 9 bits per file
  - (user, group, others) * (read, write, execute)

- Pros: all info close to object; easy to know who is allowed
- Cons: hard to store if cannot grouped; hard to know everything one can access

Capabilities

- Organized by rows: with each principal, indicate which objects may be accessed and in what ways
  - E.g., a list of <object, permission> pairs with each user, i.e., a capability list
  - Capabilities must be unforgeable and revocable
  - Capabilities are maintained completely inside OS
  - Typically, capabilities also act as names for objects; cannot even find objects not referred in one's capability list
- Example: page table for physical memory frames

- Pros: easy to know all resources one can access; combined with access
- Cons: hard to know everyone allowed to access one object; capability list is long

Access Enforcement

- Some part of the system must enforce access controls and protect authentication and authorization info
- This component has full power, so better to be small and simple

- OS is naturally the choice, but OS becomes more and more complicated
- One possible approach: security kernel
  - Limit full power to a small part in OS; rest parts only manage allowed resources
  - Example: Exokernel, SOSP 1995
  - Security kernels are still uncommon in today's OSes

- Q: What if OS is untrusted? When could this be the case?

Summary

- File basics
  - Metadata: file descriptor, a.k.a., inode, file control block
  - Operations: open/close, read/write/seek/sync
- Data block organization: which blocks for which file
  - Contiguous, linked, indexed
- Free space management: track free blocks
  - Linked lists, bitmap
- Directory and links: naming
- Performance optimization: buffer cache
- Protection: authentication and authorization

Where to Go Next For Computer Architecture

- 20470084 Computer Architecture (计算机系统结构)
  - Offered by me in the next semester, 2024 autumn
  - One of the several undergrad system courses in IIIS
- Course topics
  - Detailed processor hardware
  - Advanced cache and memory hierarchy
  - Instruction-level, data-level, and thread-level parallelism
  - Virtualization, hardware security, GPU, accelerators
  - ……
- Other system courses
  - Operating systems, databases, computer networking, distributed systems, compilers, …

The Scope of My Research Group

Innovative Data-centric Efficient Architecture Lab (IDEAL)

Domain-specific designs
- Hardware accelerators
- Software frameworks



Computation

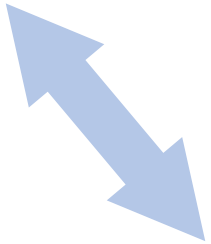




Storage

New memory architectures
- Hybrid memory
- Near-data processing

Data





Security

Hardware-assisted secure computing
- Trusted execution environments
- Cryptographic accelerators

Next Lecture: Networking (By Prof. Xu)

See you in final exam, and hopefully next semester!